

Bartłomiej Waliłko, Dawid Wypych

Wybór i implementacja modelu sztucznej inteligencji

Charakterystyka modelu cINN:

Model cINN należy do klasy odwracalnych sieci neuronowych, rozszerzając go o możliwość uwzględnienia zmiennej warunkującej. W naszym projekcie zmienną wejściową jest widmo gwiazdy (y) a zmienną wyjściową parametry fizyczne gwiazdy (x). Model uczy się rozkładu warunkowego, czyli prawdopodobieństwa parametrów gwiazdy przy danym widmie.

Model cINN (conditional Invertible Neural Network) opiera się na architekturze przepływów normalizujących (Normalizing Flows). W przeciwieństwie do standardowych sieci, które mapują dane wejściowe bezpośrednio na wynik, cINN tworzy dwukierunkową, odwracalną relację między parametrami fizycznymi a przestrzenią ukrytą (zazwyczaj o rozkładzie Gaussa), pod warunkiem (condition) zadany przez widmo gwiazdy.

Zastosowanie do danych spektroskopowych

Widmo gwiazdy zawiera informacje zakodowane w liniach absorpcyjnych i emisyjnych, które zależą od jej właściwości fizycznych. Należy wziąć pod uwagę, że różne zestawy parametrów mogą prowadzić do bardzo podobnych widm, dane obserwacyjne zawierają szum, modele fizyczne mogą być niepełne lub przybliżone. W efekcie problem rekonstrukcji parametrów jest nieliniowy i wieloznaczny (istnieje wiele możliwych rozwiązań)

Zalety cINN

Modelowanie wieloznaczności - cINN modeluje pełny rozkład, a nie pojedynczą wartość. Dzięki temu model naturalnie reprezentuje niepewność oraz uzyskuje wiele możliwych zestawów parametrów dla jednego widma.

Odwracalność i brak utraty informacji - dzięki temu możliwe jest dokładne odwzorowanie zależności między widmem a parametrami.

Dokładna estymacja rozkładu prawdopodobieństwa - cINN pozwala na bezpośrednie obliczanie funkcji wiarygodności, dzięki czemu wyniki są bardziej interpretowalne.

Oporność na szum i niepewność danych - cINN może być trenowany tak, aby uwzględniać szum w danych wejściowych, co poprawia stabilność predykcji.

Porównanie do innych modeli:

Standardowe sieci (np. CNN) mają problem, jeżeli dane są niejednoznaczne. Jeśli dwa różne zestawy parametrów dają praktycznie identyczne widmo, te sieci podadzą wartość uśrednioną, czyli błędną. cINN jest architekturą kodującą pełny rozkład prawdopodobieństwa.

Dopasowanie modeli fizycznych do szablonu - w tym przypadku problemem jest ograniczenie do oczek siatki. W przypadku, gdy gwiazda ma parametry pomiędzy węzłami, wynik jest niedokładny. Poza tym dochodzi problem dużej złożoności czasowej

Markov Chain Monte Carlo - ten model jest bardzo dobry do wyznaczania niepewności, ale jest bardzo drogi obliczeniowo.

```
import torch
import torch.nn as nn
import FrEIA.framework as Ff
import FrEIA.modules as Fm
```

```
def build_cinn(input_dim, cond_dim, n_blocks=8, hidden_dim=128):
    # Perceptron wielowarstwowy
    def subnet_fc(dims_in, dims_out):
        return nn.Sequential(
            nn.Linear(dims_in, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, dims_out)
        )

    # Inicjalizujemy z wymiarem wejściowym
    inn = Ff.SequenceINN(input_dim)

    # Dodajemy bloki sprzęgające z określeniem warunków
    for _ in range(n_blocks):
        inn.append(
            Fm.AllInOneBlock(
                cond=0, # Pierwszy warunek (index 0)
                cond_shape=(cond_dim,), # Kształt tensora warunków (w formie tupli)
                subnet_constructor=subnet_fc,
                permute_soft=False
            )
        )
    return inn
```

```
input_dim = 8575 # Ilość badanych punktów w fluxie (ilość punktów w których badamy natężenie światła)
cond_dim = 3 # Ilość warunków które przewidujemy
model = build_cinn(input_dim, cond_dim)

# Jeśli jest cuda, to łąduje na cuda
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)
print(model)
```

```
SequenceINN(
  (module_list): ModuleList(
    (0-7): 8 x AllInOneBlock(
      (softplus): Softplus(beta=0.5, threshold=20.0)
      (subnet): Sequential(
        (0): Linear(in_features=3760, out_features=128, bias=True)
        (1): ReLU()
        (2): Linear(in_features=128, out_features=7514, bias=True)
      )
    )
  )
)
```

```
from astropy.io import fits
```

```
hdul = fits.open("mwmStar-0.6.0-85995134.fits")
hdul[3].data["FLUX"][0]
```

```
array([0., 0., 0., ..., 0., 0., 0.], shape=(8575,), dtype='>f4')
```